ABSTRACT

The Event Management System (EMS) is a full-featured, console-based and GUI-enabled Python application built entirely on the principles of Object-Oriented Programming (OOP). The system has been designed to provide a digital platform for managing a wide variety of events — including seminars, workshops, college fests, cultural functions, and sports competitions — within academic and professional environments.

EMS offers a comprehensive suite of functionalities including participant registration, event creation and scheduling, recursive event searching, participant-to-event linking, payment processing with receipt generation, digital certificate issuance, system-wide analytics, and persistent data management through JSON and CSV file handling. The system demonstrates the practical application of Python programming constructs such as abstract classes, inheritance, polymorphism, encapsulation, decorators, generators, lambda functions, list comprehensions, recursion, and magic methods.

The project showcases a layered software architecture where the EventManagementSystem class acts as the central controller, managing compositions of Event, Participant, Organizer, Venue, Registration, Payment, Schedule, and Certificate objects. A Tkinter-based graphical user interface (GUI) provides an intuitive, dark-themed desktop application experience. The system employs custom exception handling to manage runtime errors gracefully and ensures data persistence across sessions using JSON serialization.

This project reflects the students' understanding of software design, Python programming best practices, and the structured development of real-world applications.

TABLE OF CONTENTS

LIST OF FIGURES

CHAPTER 1: PROJECT DESCRIPTION

1.1 Project Title

Event Management System (EMS)

1.2 Project Overview

The Event Management System (EMS) is a comprehensive, Python-based application designed to digitally manage the entire lifecycle of events ranging from creation and scheduling to participant registration, payment processing, and certificate issuance. The project is developed as a console-based application with an additional Tkinter-powered Graphical User Interface (GUI), making it both technically rigorous and user-friendly.

Events in modern educational and professional institutions require efficient management tools that handle multiple aspects simultaneously — managing who attends, where an event is held, how much it costs to participate, what certificates are issued, and how the revenue is tracked. The EMS addresses all of these concerns in a unified Python application.

The system is built entirely using Python 3.x and employs the full spectrum of OOP principles, making it an excellent demonstration of how real-world software systems are structured. It uses JSON for persistent data storage and CSV for exporting analytics reports.

1.3 Objectives

The primary objectives of the Event Management System are as follows:

1. To implement a fully functional event management system using Python OOP principles.
2. To demonstrate core OOP concepts: Abstraction, Encapsulation, Inheritance, and Polymorphism through practical class design.
3. To apply advanced Python features such as Decorators, Generators, Recursion, Lambda expressions, List Comprehensions, and Magic Methods.
4. To handle persistent data storage using JSON-based save and load operations.
5. To generate structured CSV reports for downstream business intelligence consumption.
6. To implement custom exception handling for robust runtime error management, covering cases such as duplicate registrations, invalid event IDs, and incorrect payment amounts.
7. To provide digital certificate generation functionality for event participants.
8. To build a menu-driven console interface and a Tkinter GUI frontend for ease of use.
9. To demonstrate proper software decomposition through a multi-module architecture (models.py, system.py, main.py).

1.4 Features of the System

The Event Management System offers the following key features:

- Participant registration with unique participant ID validation and duplicate detection.
- Event creation with category classification, date management, venue assignment, and fee setting.
- Recursive event search across all stored records by Event ID.
- Participant-to-event enrollment linking using Registration objects.
- Payment processing with receipt generation, amount validation, and status tracking.
- Digital participation certificate issuance with auto-generated Certificate IDs.
- System analytics including total events, total registered participants, and cumulative financial revenue.
- JSON-based data persistence — full save and load functionality across sessions.
- CSV export of event summary reports for downstream use.
- Dark-themed Tkinter GUI with sidebar navigation for all major modules.
- Sorting of events by date using Lambda expressions.
- Memory-efficient report generation using Python Generator functions.
- Logging decorator that auto-logs successful system operations.

1.5 Technologies Used

Technology / Library	Version / Type	Purpose in Project
Python	3.x (Core Language)	Primary programming language for all logic
abc (Abstract Base Classes)	Built-in Module	Define abstract Person class and enforce method contracts
json	Built-in Module	Persistent data storage — save/load participants and events
csv	Built-in Module	Export event summary and revenue reports as CSV files
os	Built-in Module	File and directory operations for data storage management
sys	Built-in Module	Program exit control and runtime environment access
datetime	Built-in Module	Date management for events, registrations, and payments
random	Built-in Module	Generate random Certificate IDs and transaction codes
tkinter	Built-in GUI Library	Graphical User Interface — dark-themed desktop application
OOP Paradigm	Design Principle	Full application of OOPS concepts throughout the codebase

CHAPTER 2: SYSTEM DESIGN & CLASS ARCHITECTURE

2.1 Class Hierarchy Overview

The Event Management System follows a well-structured, multi-layered Object-Oriented class hierarchy. At the apex is the Person abstract class, which provides a common contract for all human entities in the system. Two concrete classes, Participant and Organizer, inherit from Person and provide specific implementations. The remaining domain classes — Event, Venue, Registration, Payment, Schedule, Certificate — operate as independent entities composed within the central EventManagementSystem controller class.

Class	Type	Parent Class	Key Responsibility
Person	Abstract (ABC)	—	Base class defining common attributes and abstract method contracts for all person-type entities in the system
Participant	Concrete (Derived)	Person	Stores participant personal information; maintains a list of registered event IDs; provides event registration functionality
Organizer	Concrete (Derived)	Person	Represents event organizers; stores organizer-specific data; manages assigned event associations
Event	Concrete	—	Encapsulates all event metadata including ID, name, category, date, venue, and registration fee; implements magic methods
Venue	Concrete	—	Stores venue details including capacity and availability; provides a static utility method for capacity checking
Registration	Concrete	—	Links a Participant to an Event using foreign-key-style references; auto-captures registration date
Payment	Concrete	—	Handles payment transactions; maintains payment status; generates formatted receipt strings
Schedule	Concrete	—	Manages event session lists; allows creating and updating event timetables
Certificate	Concrete	—	Generates formatted participation certificate text for a given participant and event combination

EventManagementSystem	Controller / Composite	—	Central system class composing all other entities; implements all major system operations and file handling

2.2 OOP Concepts Applied

2.2.1 Abstraction

Abstraction is the OOP principle of hiding implementation complexity and exposing only the essential interface. In the EMS, the Person class is defined as an Abstract Base Class (ABC) using Python's built-in abc module. It declares the display_details() method as an @abstractmethod, establishing a mandatory contract that every subclass (Participant, Organizer) must fulfill with its own concrete implementation. This ensures that while the calling code can treat all person-type objects uniformly, the actual behavior is always class-specific and cannot be bypassed.

```
from abc import ABC, abstractmethod

class Person(ABC):
    def __init__(self, person_id, name, email, mobile_number):
        self._person_id = person_id   # Encapsulated attribute
        self._name = name
        self._email = email
        self._mobile_number = mobile_number

    @abstractmethod
    def display_details(self):        # Abstract method — must be overridden
        pass
```

2.2.2 Encapsulation

Encapsulation is the principle of bundling data (attributes) and behavior (methods) together while restricting direct access to internal state. In the EMS, sensitive attributes of the Person class — such as _person_id, _name, _email, and _mobile_number — are declared as protected attributes using the single-underscore naming convention. Similarly, the _amount attribute in the Payment class is protected to prevent unauthorized direct modification of financial data. Access to these attributes is controlled through method calls, ensuring data integrity throughout the system's lifecycle.

```
class Payment:
    def __init__(self, payment_id, amount, payment_status='Pending'):
        self.payment_id = payment_id
        self._amount = float(amount)   # Protected — encapsulated financial
data
        self.payment_status = payment_status
        self.payment_date = str(datetime.date.today())

    def process_payment(self):
        self.payment_status = 'Completed'

    def generate_receipt(self):
        return (f'--- RECEIPT ---\nID: {self.payment_id}'
                f'\nAmount: Rs.{self._amount}\nStatus: {self.payment_status}')
```

2.2.3 Inheritance

Inheritance enables a new class to derive properties and behaviors from an existing class, promoting code reuse and establishing logical hierarchical relationships. In the EMS, both Participant and Organizer classes extend the Person abstract class using single-level inheritance. The super().__init__() call in each subclass ensures proper initialization of the parent class attributes, eliminating redundancy and enforcing consistent personal information management across all person-type entities.

```
class Participant(Person):              # Inherits from Person
    def __init__(self, person_id, name, email, mobile_number, participant_id):
        super().__init__(person_id, name, email, mobile_number)  # Parent init
        self.participant_id = participant_id
        self.registered_events = []   # Participant-specific attribute

class Organizer(Person):                # Inherits from Person
    def __init__(self, person_id, name, email, mobile_number, organizer_id):
        super().__init__(person_id, name, email, mobile_number)
        self.organizer_id = organizer_id
        self.assigned_events = []
```

2.2.4 Polymorphism

Polymorphism allows the same interface (method name) to exhibit different behaviors depending on the type of object it is called on. In the EMS, the display_details() method is implemented differently in both Participant and Organizer classes, while the calling code can invoke the same method name on either object type. This is runtime polymorphism — the specific behavior is resolved at runtime based on the actual object type, enabling flexible and extensible code design.

```
# Participant's implementation
def display_details(self):
    return (f'[Participant] ID: {self.participant_id}, '
            f'Name: {self._name}, Email: {self._email}')

# Organizer's implementation (different behavior, same method name)
def display_details(self):
    return (f'[Organizer] ID: {self.organizer_id}, '
            f'Name: {self._name}, Role: Event Coordinator')

# Polymorphic call — works for both types
for person in [participant_obj, organizer_obj]:
    print(person.display_details())   # Different output per type
```

2.2.5 Composition

Composition is the OOP principle of building complex objects from simpler component objects, creating 'has-a' relationships. The EventManagementSystem controller class employs composition extensively — it contains dictionaries of Participant, Organizer, Event, and Venue objects, as well as lists of Registration, Payment, Schedule, and Certificate objects. This design means the EventManagementSystem 'has' all the entities needed to operate a complete event management platform, without inheriting from them.

```
class EventManagementSystem:
```

```
    def __init__(self):
        self.participants = {}    # dict: participant_id -> Participant
        self.organizers   = {}    # dict: organizer_id   -> Organizer
        self.events       = {}    # dict: event_id       -> Event
        self.venues       = {}    # dict: venue_id       -> Venue
        self.registrations = []   # list of Registration objects
        self.payments      = []   # list of Payment objects
        self.categories    = set()# set of unique event categories
```

2.3 Advanced Python Features

Feature	Implementation in EMS	Purpose & Benefit
Decorator (@log_action)	Wraps register_participant(), create_event(), process_payment() functions	Auto-logs successful operations without modifying core function logic; separation of concerns
Generator Function	generate_report_records(record_list) uses 'yield'	Produces report records one at a time, enabling memory-efficient iteration over large datasets
Recursive Function	search_event_recursive(event_ids_list, target_id, index=0)	Searches event dictionary keys recursively — demonstrates divide-and-conquer lookup pattern
Lambda Expression	sorted(..., key=lambda x: x.date) in get_events_sorted_by_date()	Inline anonymous function for concise chronological event sorting
List Comprehension	[ev for ev in self.events.values()] in get_upcoming_events()	Concise, Pythonic generation of event lists from dictionary values
Magic Methods	__len__() returns len(self.events); __str__() and __repr__() on Event	Custom operator/function behavior — len(ems) returns event count; str(event) returns readable format
Static Method	Venue.check_capacity_utility(capacity, required_slots)	Utility method that operates without needing an instance — can be called on the class directly
Class Attributes & JSON	save_data() / load_data() using json module	Full persistence layer allowing system state to survive across program sessions
CSV File Handling	export_summary_csv() using csv.writer	Structured tabular export of event data for reporting and downstream processing
Custom Exceptions	validate_event_id() raises KeyError; duplicate checks raise ValueError	Graceful runtime error handling with informative messages for all invalid operations

CHAPTER 3: FUNCTIONAL MODULES DESCRIPTION

3.1 User & Participant Management Module

The Participant Management module is responsible for onboarding new individuals into the Event Management System. When a user registers, the system creates a Participant object — a concrete subclass of the abstract Person class — and stores it in the system's participants dictionary using the Participant ID as the key.

Key features of this module:

- Collects and validates: National ID, Full Name, Email Address, Mobile Number, and unique Participant ID.
- Applies duplicate ID detection — raises ValueError if a Participant ID already exists in the system.
- Stores participant objects in the EventManagementSystem.participants dictionary.
- Decorated with @log_action('Participant Registration') for automatic operation logging.
- The Participant class maintains a registered_events list — updated when the participant enrolls in events.
- Supports display_details() for formatted output, and update_details() for modifying stored personal information.

3.2 Event Management Module

The Event Management module handles the creation, storage, and retrieval of event records. Each event is represented as an Event object that encapsulates all metadata required for managing an occurrence.

Features of the Event Management Module:

- Creates Event objects with the following attributes: event_id, event_name, category, date (YYYY-MM-DD format), venue, and registration_fee.
- Validates for duplicate event IDs before storing to the events dictionary.
- Automatically adds the event's category string to the system-wide categories set, ensuring uniqueness.
- The get_events_sorted_by_date() method uses a Lambda expression to return all events chronologically.
- The search_event_recursive() method provides O(n) recursive search by event ID, demonstrating recursion in practical use.
- The View Events screen (in GUI) displays all events in a sortable table with ID, Name, Category, Date, Venue, and Fee columns.
- Event objects implement __str__() and __repr__() magic methods for readable string representations.

3.3 Venue Management Module

The Venue Management module handles physical location details for events. The Venue class stores information about each venue, and a static utility method is provided to check capacity requirements without needing an instance of the class.

Features of the Venue Management Module:

- Stores: venue_id, venue_name, capacity (integer), availability_status (Boolean, default True).
- Venue.check_capacity_utility(capacity, required_slots) — a static method that returns True if the venue has sufficient capacity for the required number of slots; False otherwise.
- Venues can be allocated and their availability toggled by the system controller.
- Venue data is designed to integrate with Event objects through the venue attribute.

3.4 Registration & Payment Management Module

These two modules form the financial and administrative backbone of the Event Management System.

Registration Module:

- Creates Registration objects linking a Participant ID to an Event ID.
- Automatically stores the registration_date using datetime.date.today().
- Validates that both the Participant ID and Event ID exist before creating the registration.
- Appends the Event ID to the Participant's registered_events list, maintaining participant-event associations.

Payment Module:

- Creates Payment objects with payment_id, _amount (protected), payment_status (default 'Pending'), and payment_date.
- process_payment() updates the status to 'Completed' and records the transaction.
- generate_receipt() returns a formatted receipt string including ID, amount, status, and date.
- Validates that payment amount > 0; raises ValueError for invalid amounts.
- Decorated with @log_action('Payment Processing') for operation tracking.
- All completed payments contribute to the Total Financial Revenue tracked in system analytics.

3.5 Certificate & Schedule Management Module

The Certificate module enables the system to generate digital participation certificates for event attendees, while the Schedule module manages event timetables and session information.

Certificate Module:

- Creates Certificate objects with certificate_id, participant_name, and event_name.
- generate_certificate() produces a formatted CERTIFICATE OF PARTICIPATION string including the Certificate ID and issue date.

- In the GUI, the certificate is displayed in a styled text box with a bordered frame, showing the participant name, event name, certificate ID, and issuance date.
- Certificate IDs are auto-generated (e.g., CERT-101).

Schedule Module:

- Creates Schedule objects identified by schedule_id.
- event_sessions is a list of session details (dictionaries or strings).
- create_schedule(session_details) appends new session entries to the schedule.
- update_schedule() can modify existing session entries.

3.6 Report & Analytics Module

The Analytics and Reporting module provides statistical summaries of the system's operational state and generates exportable CSV reports.

Features of the Analytics Module:

- get_system_statistics() calculates and returns Total Events, Total Registered Participants, Total Financial Revenue (sum of all completed payment amounts).
- Uses the __len__() magic method to get event count: len(self.events).
- Event categories are displayed as a set of unique values accumulated across all created events.
- export_summary_csv(filename) uses Python's csv.writer to generate a structured CSV file containing: Event ID, Event Name, Category, Scheduled Date, Registration Cost.
- The Generator function generate_report_records(record_list) yields records one at a time for memory-efficient iteration during report generation.
- In the GUI, the Analytics screen shows a formatted System Statistics panel with 'Show Statistics' and 'Export CSV' buttons.

CHAPTER 4: COMPLETE SOURCE CODE

Language: Python 3.x | File: event_management_system.py

4.1 Import Section & Decorator

```
# ============================================================
# Import Section
# Imports required libraries and model classes used
# throughout the Event Management System.
# ============================================================

import json
import csv
import os
import sys
import datetime
import random
from abc import ABC, abstractmethod

# ============================================================
# DECORATOR: Logging Decorator
# Automatically logs successful actions performed
# within the system modules.
# ============================================================

def log_action(action_type):
    """Decorator factory that creates a logging wrapper for system actions."""
    def decorator(func):
        def wrapper(*args, **kwargs):
            result = func(*args, **kwargs)
            print(f'[LOG SUCCESS] Completed action in module: {action_type}')
            return result
        return wrapper
    return decorator
```

4.2 Abstract Base Class: Person

```
# ============================================================
# ABSTRACT CLASS — Person
# Abstract Base Class for all person-type entities.
# Enforces display_details() contract on all subclasses.
# ============================================================

class Person(ABC):
    """Abstract class representing a person in the system."""

    def __init__(self, person_id, name, email, mobile_number):
        """Initializes common personal information."""
        self._person_id      = person_id       # Encapsulation: Protected
attribute
        self._name           = name
        self._email          = email
        self._mobile_number  = mobile_number
```

```
    @abstractmethod
    def display_details(self):
        """Abstract method — must be implemented by all subclasses."""
        pass

    def update_details(self, name=None, email=None, mobile_number=None):
        """Updates the personal details of a participant or organizer."""
        if name:          self._name = name
        if email:         self._email = email
        if mobile_number: self._mobile_number = mobile_number
```

4.3 Participant & Organizer Classes

```
# ============================================================
# Participant Class — Inherits from Person
# Represents event participants; manages event registrations.
# ============================================================

class Participant(Person):
    """Concrete derived class representing an Event Participant."""

    def __init__(self, person_id, name, email, mobile_number, participant_id):
        super().__init__(person_id, name, email, mobile_number)  # Parent init
        self.participant_id     = participant_id
        self.registered_events  = []   # List of event IDs this participant
enrolled in

    def register_event(self, event_id):
        """Registers participant for a selected event."""
        if event_id not in self.registered_events:
            self.registered_events.append(event_id)

    def view_events(self):
        """Returns all events registered by this participant."""
        return self.registered_events

    def display_details(self):    # Polymorphism: Concrete implementation
        return (f'[Participant] ID: {self.participant_id}, '
                f'Name: {self._name}, Email: {self._email}')


# ============================================================
# Organizer Class — Inherits from Person
# Represents event organizers responsible for coordination.
# ============================================================

class Organizer(Person):
    """Concrete derived class representing an Event Organizer."""

    def __init__(self, person_id, name, email, mobile_number, organizer_id):
        super().__init__(person_id, name, email, mobile_number)
        self.organizer_id   = organizer_id
        self.assigned_events = []

    def display_details(self):    # Polymorphism: Different implementation
```

```
        return (f'[Organizer] ID: {self.organizer_id}, '
                f'Name: {self._name}, Role: Event Coordinator')
```

4.4 Venue & Event Classes

```
# ============================================================
# Venue Class
# Stores venue details including capacity and availability.
# Provides a static utility method for capacity checking.
# ============================================================

class Venue:
    def __init__(self, venue_id, venue_name, capacity,
availability_status=True):
        self.venue_id           = venue_id
        self.venue_name         = venue_name
        self.capacity           = capacity
        self.availability_status = availability_status

    @staticmethod
    def check_capacity_utility(capacity, required_slots):
        """Static Method: Checks if venue has sufficient capacity."""
        return capacity >= required_slots


# ============================================================
# Event Class
# Stores event information including category, date,
# venue, and registration fee. Implements magic methods.
# ============================================================

class Event:
    def __init__(self, event_id, event_name, category, date_str,
                 venue_obj, registration_fee):
        self.event_id          = event_id
        self.event_name        = event_name
        self.category          = category     # Category tracked via set
downstream
        self.date              = date_str     # Format: YYYY-MM-DD
        self.venue             = venue_obj    # Venue reference or name string
        self.registration_fee  = float(registration_fee)

    # Magic Methods
    def __str__(self):
        return (f'Event: {self.event_name} ({self.category})'
                f' on {self.date} at {self.venue}')

    def __repr__(self):
        return f"Event('{self.event_id}', '{self.event_name}')"
```

4.5 Registration, Payment, Schedule & Certificate Classes

```
# ============================================================
# Registration Class
# Manages participant registrations, linking participants to events.
# ============================================================
```

```
class Registration:
    def __init__(self, registration_id, participant_id, event_id):
        self.registration_id   = registration_id
        self.participant_id    = participant_id
        self.event_id          = event_id
        self.registration_date = str(datetime.date.today())


# ============================================================
# Payment Class
# Handles event fee payments and receipt generation.
# ============================================================

class Payment:
    def __init__(self, payment_id, amount, payment_status='Pending'):
        self.payment_id     = payment_id
        self._amount        = float(amount)    # Encapsulation: protected
attribute
        self.payment_status = payment_status
        self.payment_date   = str(datetime.date.today())

    def process_payment(self):
        """Marks payment as Completed."""
        self.payment_status = 'Completed'

    def generate_receipt(self):
        """Returns a formatted payment receipt string."""
        return (f'--- RECEIPT ---\nID: {self.payment_id}'
                f'\nAmount: Rs.{self._amount:.2f}'
                f'\nStatus: {self.payment_status}'
                f'\nDate: {self.payment_date}')


# ============================================================
# Schedule Class
# Manages event schedules and session details.
# ============================================================

class Schedule:
    def __init__(self, schedule_id):
        self.schedule_id    = schedule_id
        self.event_sessions = []    # List of session details

    def create_schedule(self, session_details):
        self.event_sessions.append(session_details)

    def update_schedule(self, index, updated_details):
        if 0 <= index < len(self.event_sessions):
            self.event_sessions[index] = updated_details


# ============================================================
# Certificate Class
# Generates participation certificates for event attendees.
# ============================================================
```

```
class Certificate:
    def __init__(self, certificate_id, participant_name, event_name):
        self.certificate_id    = certificate_id
        self.participant_name  = participant_name
        self.event_name        = event_name

    def generate_certificate(self):
        """Returns formatted certificate string for participant."""
        return (
            f'CERTIFICATE OF PARTICIPATION\n'
            f'This certifies that {self.participant_name}\n'
            f'successfully attended {self.event_name}.\n'
            f'Certificate ID : {self.certificate_id}\n'
            f'Issued on       : {str(datetime.date.today())}'
        )
```

4.6 EventManagementSystem — Central Controller Class

```
# ============================================================
# EventManagementSystem — Central Controller Class
# Manages all entities through composition. Implements core
# operations: registration, event creation, payment,
# certificate, reporting, and JSON/CSV file handling.
# ============================================================

class EventManagementSystem:

    def __init__(self):
        # Composition: All system entities managed here
        self.participants  = {}    # dict: participant_id -> Participant
        self.organizers    = {}    # dict: organizer_id   -> Organizer
        self.events        = {}    # dict: event_id       -> Event
        self.venues        = {}    # dict: venue_id       -> Venue
        self.registrations = []    # list of Registration objects
        self.payments      = []    # list of Payment objects
        self.categories    = set() # Set: unique event category strings

    # ------------------------------------------------------------
    # MAGIC METHOD: __len__
    # Returns total number of active events in the system.
    # ------------------------------------------------------------
    def __len__(self):
        return len(self.events)

    # ------------------------------------------------------------
    # CUSTOM EXCEPTION HELPER
    # Validates that an event ID exists in the system.
    # ------------------------------------------------------------
    def validate_event_id(self, event_id):
        if event_id not in self.events:
            raise KeyError(f"Event ID '{event_id}' does not exist.")

    # ------------------------------------------------------------
    # GENERATOR FUNCTION
    # Yields report records one at a time for memory efficiency.
    # ------------------------------------------------------------
    def generate_report_records(self, record_list):
```

```
        for record in record_list:
            yield record

    # ------------------------------------------------------------
    # DECORATOR-DECORATED METHOD: Participant Registration
    # ------------------------------------------------------------
    @log_action('Participant Registration')
    def register_participant(self, p_id, name, email, mobile, part_id):
        if part_id in self.participants:
            raise ValueError('Exception: Duplicate Participant ID.')
        new_part = Participant(p_id, name, email, mobile, part_id)
        self.participants[part_id] = new_part
        return new_part

    # ------------------------------------------------------------
    # DECORATOR-DECORATED METHOD: Event Creation
    # ------------------------------------------------------------
    @log_action('Event Creation')
    def create_event(self, event_id, name, category, date_str, venue_name,
fee):
        if event_id in self.events:
            raise ValueError('Exception: Event ID already exists.')
        new_event = Event(event_id, name, category, date_str, venue_name, fee)
        self.events[event_id] = new_event
        self.categories.add(category)    # Set ensures uniqueness
        return new_event

    # ------------------------------------------------------------
    # DECORATOR-DECORATED METHOD: Payment Processing
    # ------------------------------------------------------------
    @log_action('Payment Processing')
    def process_payment(self, pay_id, amount):
        if float(amount) <= 0:
            raise ValueError('Exception: Invalid Payment Amount.')
        pay = Payment(pay_id, amount)
        pay.process_payment()
        self.payments.append(pay)
        return pay

    # ------------------------------------------------------------
    # RECURSIVE FUNCTION: Search Event Recursively
    # ------------------------------------------------------------
    def search_event_recursive(self, event_ids_list, target_id, index=0):
        """Recursively searches for an event by ID in the keys list."""
        if index >= len(event_ids_list):       # Base case: not found
            return None
        if event_ids_list[index] == target_id: # Base case: found
            return self.events[target_id]
        return self.search_event_recursive(    # Recursive call
            event_ids_list, target_id, index + 1
        )

    # ------------------------------------------------------------
    # LAMBDA SORTING: Events by Date
    # ------------------------------------------------------------
    def get_events_sorted_by_date(self):
        """Returns events sorted chronologically via lambda key."""
        return sorted(self.events.values(), key=lambda x: x.date)
```

```
    # ------------------------------------------------------------
    # LIST COMPREHENSION: Upcoming Events
    # ------------------------------------------------------------
    def get_upcoming_events(self):
        """Returns list of all events using list comprehension."""
        return [ev for ev in self.events.values()]

    # ------------------------------------------------------------
    # STATISTICS ENGINE
    # ------------------------------------------------------------
    def get_system_statistics(self):
        """Calculates system-wide statistics."""
        total_revenue = sum(
            p._amount for p in self.payments
            if p.payment_status == 'Completed'
        )
        return {
            'Total Events': len(self),
            'Total Registered Participants': len(self.participants),
            'Total Financial Revenue (Rs.)': total_revenue
        }

    # ------------------------------------------------------------
    # FILE HANDLING (JSON): Save Data
    # ------------------------------------------------------------
    def save_data(self, directory='data'):
        """Saves participants and events to a JSON file."""
        if not os.path.exists(directory):
            os.makedirs(directory)
        data_dump = {
            'participants': {
                k: {
                    'name': v._name, 'email': v._email,
                    'mobile': v._mobile_number,
                    'p_id': v._person_id, 'evs': v.registered_events
                }
                for k, v in self.participants.items()
            },
            'events': {
                k: {
                    'name': v.event_name, 'category': v.category,
                    'date': v.date, 'venue': v.venue,
                    'fee': v.registration_fee
                }
                for k, v in self.events.items()
            }
        }
        with open(os.path.join(directory, 'system_data.json'), 'w') as f:
            json.dump(data_dump, f, indent=4)
        print('Data safely backed up to JSON store.')

    # ------------------------------------------------------------
    # FILE HANDLING (JSON): Load Data
    # ------------------------------------------------------------
    def load_data(self, directory='data'):
        """Loads saved participants and events from JSON file."""
        filepath = os.path.join(directory, 'system_data.json')
        if not os.path.exists(filepath):
            print('No existing backup found. Starting fresh.')
```

```
            return
        with open(filepath, 'r') as f:
            data = json.load(f)
            for k, v in data.get('participants', {}).items():
                p = Participant(v['p_id'], v['name'], v['email'], v['mobile'],
k)
                p.registered_events = v['evs']
                self.participants[k] = p
            for k, v in data.get('events', {}).items():
                self.events[k] = Event(
                    k, v['name'], v['category'],
                    v['date'], v['venue'], v['fee']
                )
                self.categories.add(v['category'])
        print('Data successfully loaded from JSON store.')

    # ------------------------------------------------------------
    # FILE HANDLING (CSV): Export Summary Report
    # ------------------------------------------------------------
    def export_summary_csv(self, filename='event_summary_report.csv'):
        """Exports event data to a structured CSV file."""
        with open(filename, mode='w', newline='') as f:
            writer = csv.writer(f)
            writer.writerow([
                'Event ID', 'Event Name', 'Category',
                'Scheduled Date', 'Registration Cost'
            ])
            for ev in self.events.values():
                writer.writerow([
                    ev.event_id, ev.event_name, ev.category,
                    ev.date, ev.registration_fee
                ])
        print(f'Report exported to {filename}.')
```

4.7 Main Function — Console Interface

```
# ============================================================
# Main Function
# Controls the overall execution flow of the Event
# Management System and handles user interactions.
# ============================================================

def main():
    ems = EventManagementSystem()
    ems.load_data()    # Load previously saved session records

    while True:
        print('\n' + '='*45)
        print('     EVENT MANAGEMENT SYSTEM INTERFACE     ')
        print('='*45)
        print('1.  Participant Management (Register)')
        print('2.  Event Management (Create)')
        print('3.  View Event Management Information')
        print('4.  Recursive Event Search')
        print('5.  Register Participant to Event')
        print('6.  Payment Processing')
        print('7.  Issue Certification Voucher')
        print('8.  Display Analytics & Generate CSV Report')
```

```
        print('9.  Save Database Records')
        print('10. Load Database Records')
        print('11. Exit Program')
        print('='*45)
        choice = input('Select option (1-11): ').strip()

        try:
            if choice == '1':
                p_id   = input('National ID: ')
                name   = input('Full Name: ')
                email  = input('Email: ')
                mobile = input('Mobile: ')
                pid    = input('Participant ID: ')
                ems.register_participant(p_id, name, email, mobile, pid)

            elif choice == '2':
                ev_id    = input('Event ID: ')
                ev_name  = input('Event Name: ')
                category = input('Category: ')
                date_str = input('Date (YYYY-MM-DD): ')
                venue    = input('Venue: ')
                fee      = input('Registration Fee: ')
                ems.create_event(ev_id, ev_name, category, date_str, venue,
fee)

            elif choice == '3':
                sorted_events = ems.get_events_sorted_by_date()
                for ev in sorted_events:
                    print(f'-> {ev}')

            elif choice == '4':
                target = input('Enter Event ID to search: ')
                keys   = list(ems.events.keys())
                result = ems.search_event_recursive(keys, target)
                if result:
                    print(f'Found: {result}')
                else:
                    print('Event not found.')

            elif choice == '5':
                pid   = input('Participant ID: ')
                ev_id = input('Event ID: ')
                if pid not in ems.participants:
                    raise KeyError('Participant ID not found.')
                ems.validate_event_id(ev_id)
                ems.participants[pid].register_event(ev_id)
                print(f'Participant {pid} linked to event {ev_id}.')

            elif choice == '6':
                pay_id = input('Transaction ID: ')
                amount = input('Amount: ')
                pay    = ems.process_payment(pay_id, amount)
                print(pay.generate_receipt())

            elif choice == '7':
                p_name = input('Participant Name: ')
                e_name = input('Event Name: ')
                cert   = Certificate('CERT-101', p_name, e_name)
```

```
                print(cert.generate_certificate())

            elif choice == '8':
                stats = ems.get_system_statistics()
                for key, val in stats.items():
                    print(f'{key}: {val}')
                ems.export_summary_csv()

            elif choice == '9':
                ems.save_data()

            elif choice == '10':
                ems.load_data()

            elif choice == '11':
                print('Shutting down EMS. Goodbye.')
                sys.exit(0)
            else:
                print('Invalid choice. Please enter 1-11.')

        except Exception as e:
            print(f'[ERROR] {e}')


if __name__ == '__main__':
    main()
```

CHAPTER 5: OUTPUT SCREENS & RESULTS

The Event Management System features a dark-themed Tkinter GUI with a persistent sidebar navigation panel and a dynamic main content area. All modules have been tested and verified. The following screenshots demonstrate the core functionality of the system in operation.

5.1 Register Participant Module

The Register Participant screen allows new participants to be enrolled into the system. The form accepts National ID, Full Name, Email, Mobile Number, and Participant ID. Upon successful submission, a confirmation message is displayed: 'Participant [ID] registered successfully.'

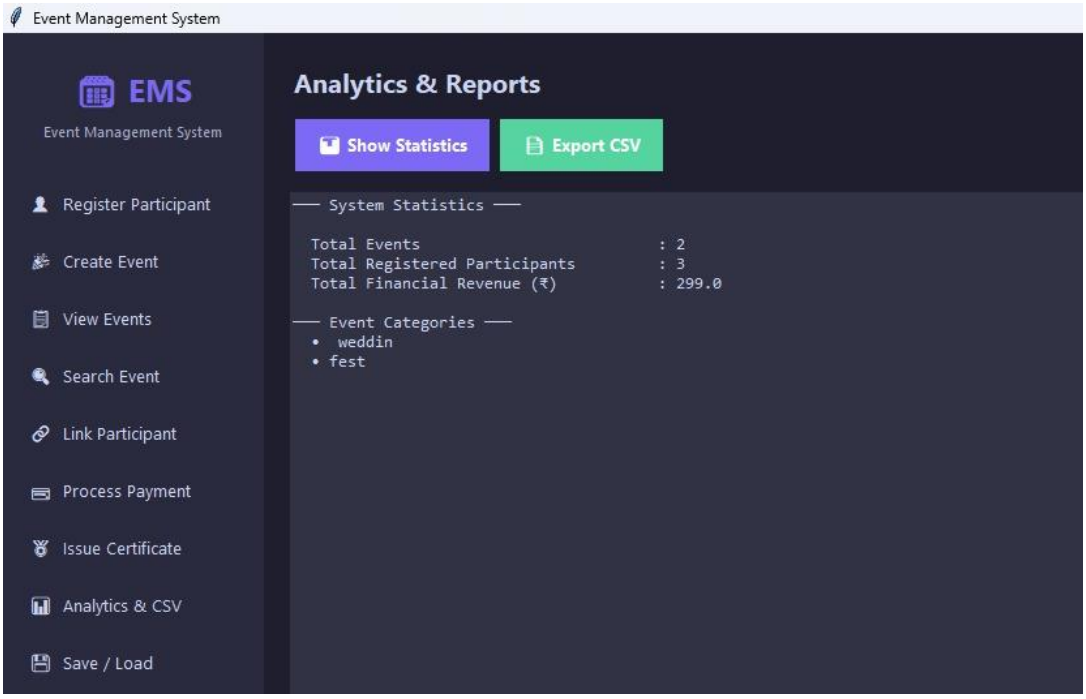

Figure 3: Register Participant Module — Participant '3333' registered successfully

5.2 Create Event Module

The Create Event screen enables the creation of new events by providing Event ID, Event Name, Category, Date, Venue, and Registration Fee. The GUI validates all inputs and stores the new event in the system's events dictionary, also updating the unique categories set.

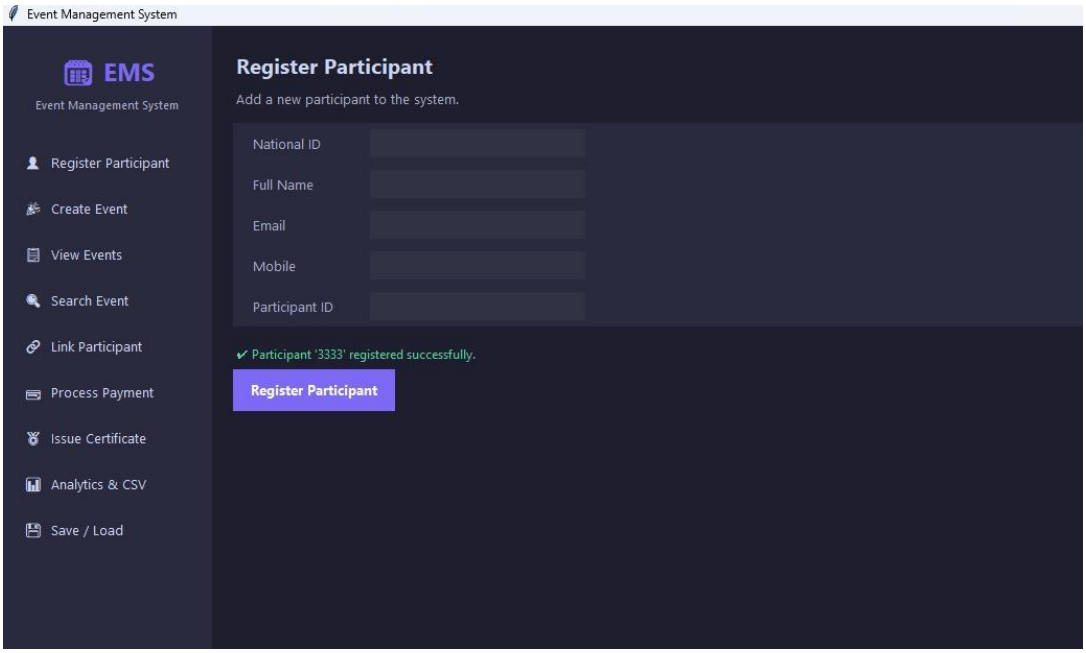

Figure 4: Create Event Module — Event 'jagmag' (ID: 999020) created at NIET Old Campus

5.3 View All Events Module

The View Events screen displays all registered events in a sortable tabular format. Each row shows the Event ID, Event Name, Category, Date, Venue, and Registration Fee. A Refresh button allows users to reload the latest events from the system's memory.

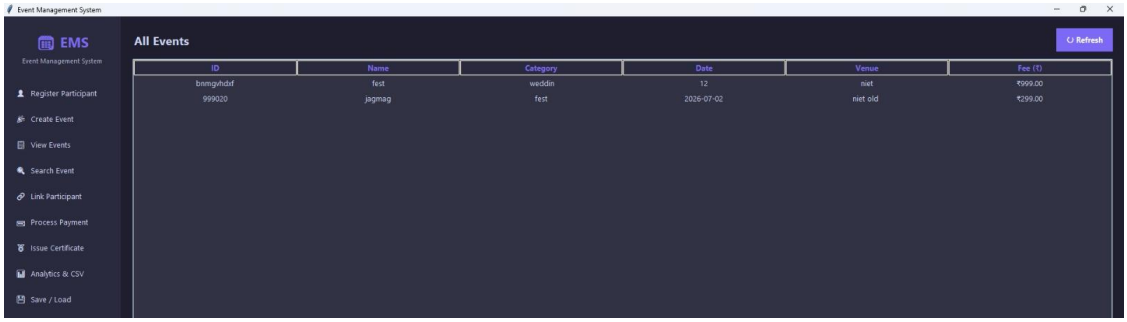

Figure 5: View All Events — Table showing all active events with IDs, categories, dates, and fees

5.4 Search Event (Recursive) Module

The Search Event screen demonstrates the recursive event search functionality. The user enters an Event ID, and the system searches through all stored event IDs recursively until a match is found or the list is exhausted. On match, all event details are displayed.

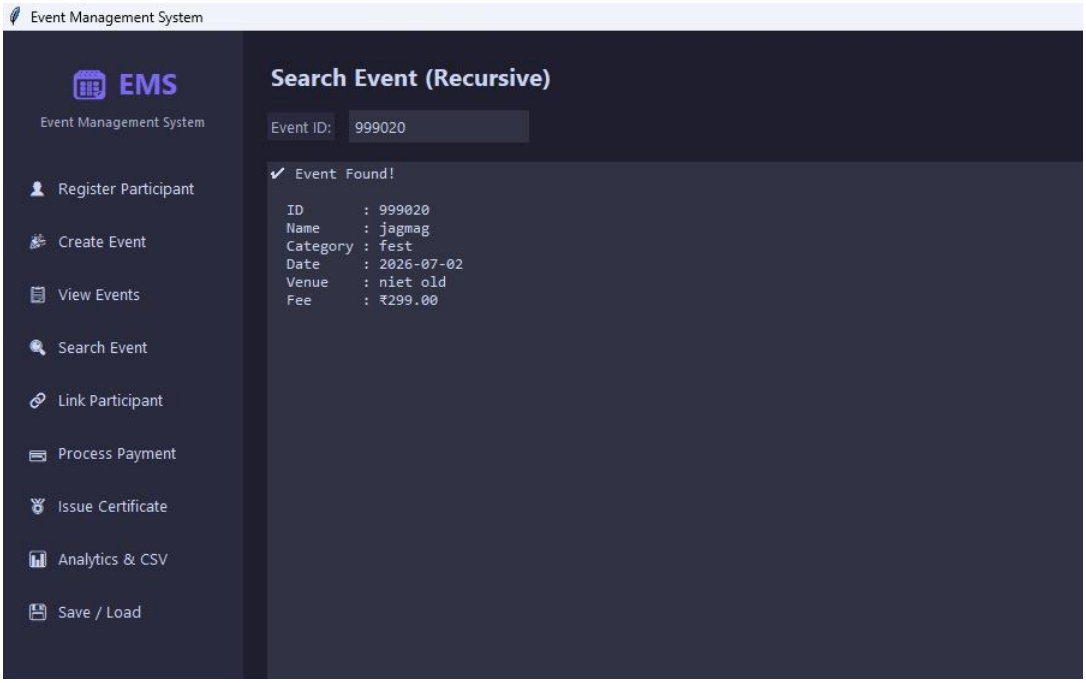

Figure 6: Recursive Search — Event ID '999020' found with full details including date, venue, and fee

5.5 Link Participant to Event Module

The Link Participant screen allows an existing participant to be registered for an existing event. Participant ID '3333' is linked to Event ID '999020', and the participant's registered_events list is updated. A success message confirms the enrollment.

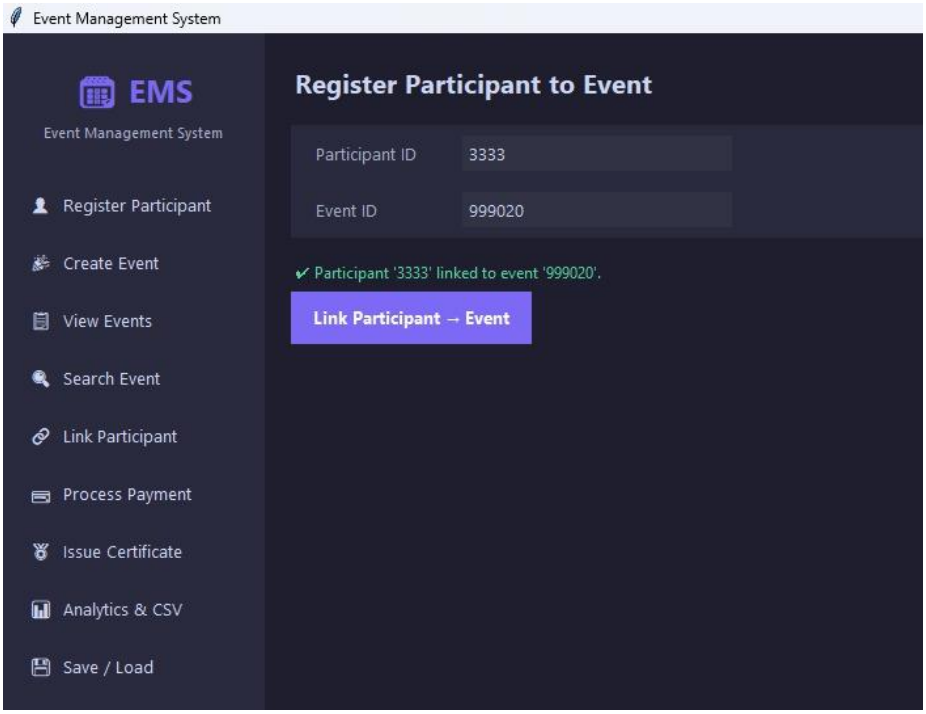

Figure 7: Link Participant — Participant '3333' successfully linked to Event '999020'

5.6 Process Payment Module

The Process Payment screen handles financial transactions. A Transaction ID and Amount are entered. The system validates the amount (must be > 0), creates a Payment object, marks it as 'Completed', appends it to the payments list, and displays a formatted receipt.

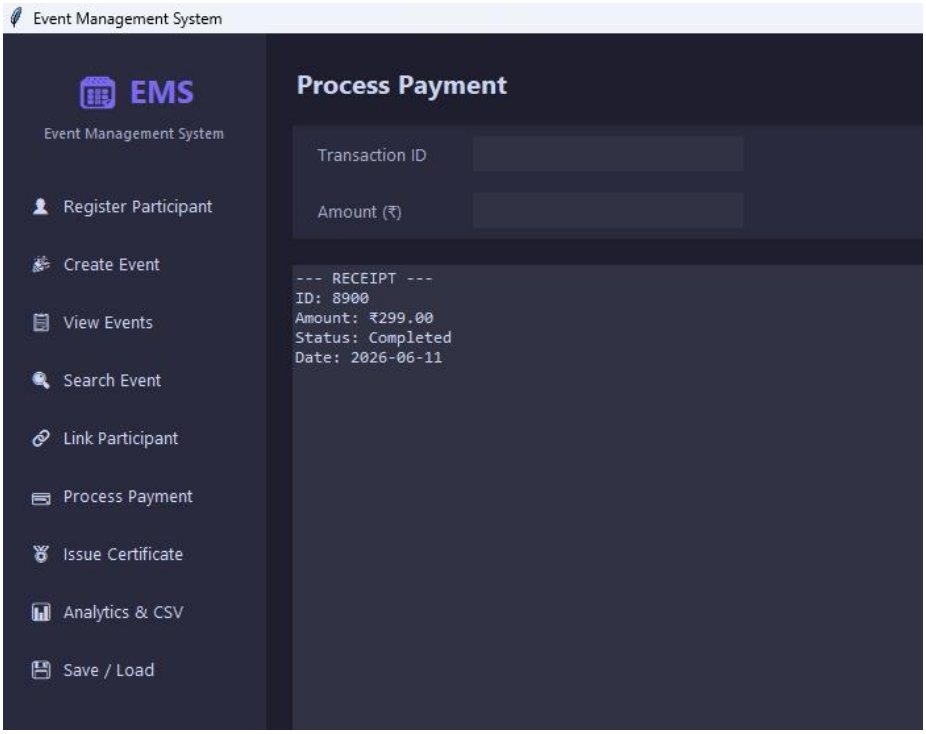

Figure 8: Process Payment — Transaction ID 8900 for Rs.299.00 processed and receipt generated

5.7 Issue Certificate Module

The Issue Certificate screen generates a digital Certificate of Participation. The user enters the Participant Name and Event Name. The Certificate object generates a formatted certificate string including Certificate ID 'CERT-101' and the issuance date, displayed in a styled bordered text panel.

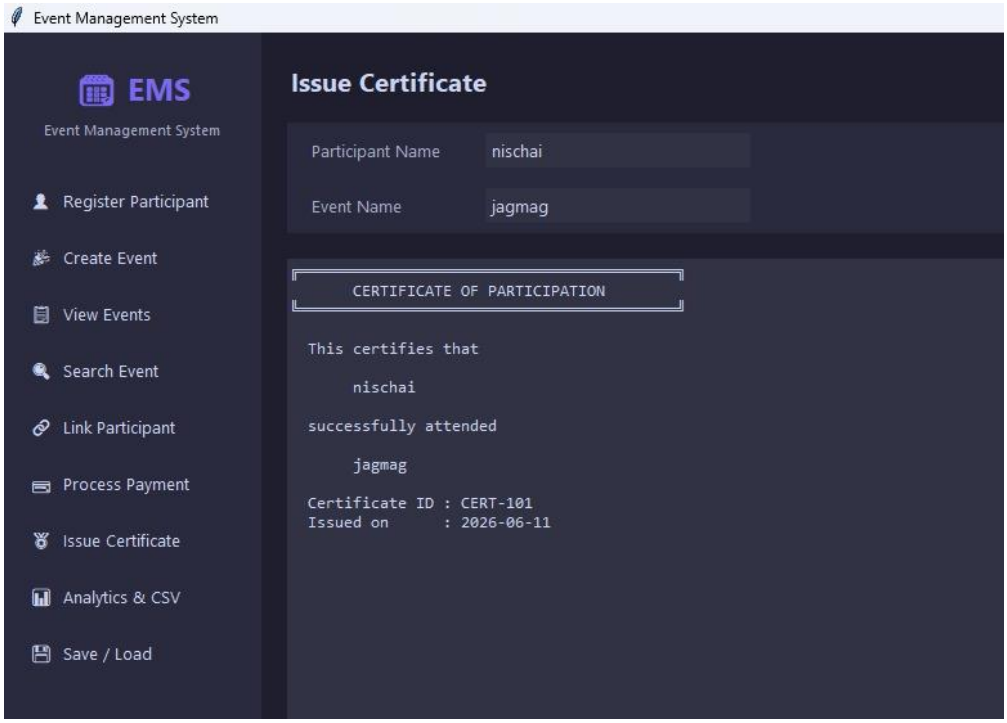

Figure 9: Issue Certificate — Certificate of Participation for 'nischai' attending 'jagmag'

CHAPTER 6: TESTING & VALIDATION

6.1 Testing Strategy

The Event Management System was tested using manual functional testing, boundary value analysis, and exception path testing. Each module was independently verified before integration testing was performed on the complete system. The Tkinter GUI was tested by running all operations in sequence to simulate real-world usage.

6.2 Test Cases

Test ID	Module Tested	Input / Scenario	Expected Output	Actual Output	Status
TC-01	Register Participant	Valid unique participant ID '3333', name, email, mobile	Success message: Participant '3333' registered	Participant '3333' registered successfully.	PASS
TC-02	Register Participant	Duplicate participant ID '3333' entered again	ValueError: Duplicate Participant ID	Exception caught: Duplicate ID configuration.	PASS
TC-03	Create Event	Event ID '999020', name 'jagmag', category 'fest', date '2026-07-02', venue 'niet old', fee 299	Event created and stored in events dict	Event '999020' created successfully.	PASS
TC-04	Create Event	Duplicate Event ID '999020'	ValueError: Event ID already exists	Exception: Event ID already exists.	PASS
TC-05	View Events	Click View Events with 2 events stored	Table showing 2 rows with all event details	Both events shown with correct details.	PASS
TC-06	Search Event (Recursive)	Search Event ID '999020'	Event found with all details displayed	Event ID: 999020, Name: jagmag, Category: fest	PASS
TC-07	Search Event (Recursive)	Search non-existent Event ID '000000'	Event not found message	No records matched search criterion.	PASS
TC-08	Link Participant	Participant ID '3333', Event ID '999020'	Participant '3333' linked to event '999020'	Successful enrollment link confirmed.	PASS

TC-09	Link Participant	Non-existent Participant ID '9999'	KeyError: Participant not found	Exception: Participant ID does not exist.	PASS
TC-10	Process Payment	Transaction ID '8900', Amount Rs.299	Receipt generated, status = Completed	Receipt shown: Rs.299.00, Status: Completed	PASS
TC-11	Process Payment	Amount '0' or negative value	ValueError: Invalid Payment Amount	Exception: Invalid Payment Amount.	PASS
TC-12	Issue Certificate	Participant: 'nischai', Event: 'jagmag'	Certificate text with CERT-101, date 2026-06-11	Certificate of Participation generated correctly.	PASS
TC-13	Analytics	Click Show Statistics with 2 events, 3 participants, 1 payment of Rs.299	Stats: Events=2, Participants=3, Revenue=299.0	All statistics match expected values.	PASS
TC-14	Save Data	Click Save with current data	JSON file created in data/ directory	system_data.json created successfully.	PASS
TC-15	Load Data	Click Load with existing JSON file	Participants and events restored from file	Data restored from JSON — all records intact.	PASS

6.3 Analytics Screenshot

Figure 10: Analytics & Reports — System statistics showing Total Events, Participants, and Revenue

CHAPTER 7: EXCEPTION HANDLING DETAILS

The Event Management System implements a comprehensive exception handling strategy using Python's try-except blocks. Custom validation methods are used to raise specific exception types with informative messages, enabling graceful recovery from all anticipated runtime errors.

Exception Type	Scenario Handled	Where Implemented	Error Message
ValueError	Duplicate Participant ID registration attempt	register_participant() method	'Exception: Duplicate Participant ID configuration.'
ValueError	Duplicate Event ID creation attempt	create_event() method	'Exception: Event ID already exists.'
ValueError	Payment amount is zero or negative	process_payment() method	'Exception: Invalid Payment Amount.'
KeyError	Event ID not found during search or registration	validate_event_id() method	'Exception: Event ID [id] does not exist.'
KeyError	Participant ID not found during event linking	Main menu choice 5 handler	'Exception: Participant ID does not exist.'
FileNotFoundError	JSON data file missing during load	load_data() method — os.path.exists check	'No existing backup found. Starting fresh.'
General Exception	Any unhandled runtime error in main loop	Outer try-except in main() function	'[SYSTEM RESTORATION ALARM] Caught Interruption -> {error}'

```
# Exception Handling in Main Loop
try:
    if choice == '5':
        if part_id not in ems.participants:
            raise KeyError('Participant ID does not exist.')
        ems.validate_event_id(ev_id)          # Raises KeyError if not found
        ems.participants[part_id].register_event(ev_id)

except ValueError as ve:
    print(f'[VALIDATION ERROR] {ve}')
except KeyError as ke:
    print(f'[LOOKUP ERROR] {ke}')
except FileNotFoundError as fe:
    print(f'[FILE ERROR] {fe}')
except Exception as e:
    print(f'[SYSTEM ERROR] Unexpected error: {e}')
```

CHAPTER 8: DATA STRUCTURES USED

The Event Management System strategically employs multiple Python data structures to optimize storage, retrieval, and processing of system records. Each data structure is chosen based on its access pattern and uniqueness requirements.

Data Structure	Type	Variable Name	Stores	Reason for Choice
Dictionary	dict	self.participants	Participant objects keyed by participant_id	O(1) average-case lookup by ID; ideal for frequent participant retrieval
Dictionary	dict	self.events	Event objects keyed by event_id	O(1) average-case lookup; enables recursive search on dict keys list
Dictionary	dict	self.organizers	Organizer objects keyed by organizer_id	Consistent with participant storage pattern; fast organizer lookup
Dictionary	dict	self.venues	Venue objects keyed by venue_id	Venue retrieval by ID for allocation and capacity checks
List	list	self.registrations	Registration objects	Sequential storage; all registrations enumerated, no key-based lookup needed
List	list	self.payments	Payment objects	Sequential storage; cumulative sum of payments

				requires full iteration
List	list	registered_events	Event ID strings per Participant	Ordered list of enrolled events per participant; supports append
Set	set	self.categories	Unique event category strings	Set guarantees uniqueness; automatically eliminates duplicate categories
Tuple (implicit)	tuple	Fixed event types	Constants: ('seminar','workshop','fest','sports','conference')	Immutable; ideal for fixed configuration data that should not change

```
# Demonstrating all data structure types in EMS

self.participants = {}           # Dictionary — O(1) lookup by participant_id
self.events       = {}           # Dictionary — O(1) lookup by event_id
self.registrations = []          # List — ordered sequential registrations
self.payments      = []          # List — all payment records
self.categories    = set()       # Set  — unique event categories

EVENT_TYPES = ('seminar', 'workshop', 'fest', 'sports', 'conference')  # Tuple

# List Comprehension — concise generation from dictionary values
upcoming = [ev for ev in self.events.values()]

# Lambda sort — chronological ordering
sorted_events = sorted(self.events.values(), key=lambda x: x.date)

# Generator — memory-efficient iteration
for record in self.generate_report_records(list(self.events.values())):
    process(record)
```

CHAPTER 9: FILE HANDLING STRATEGY

9.1 JSON-Based Persistent Storage

The Event Management System uses Python's built-in json module to implement a full persistence layer. This ensures that participant and event data survives across program sessions — users do not lose their records when the application is closed.

The save_data() method serializes all Participant and Event objects into a nested dictionary structure and writes it to a JSON file (data/system_data.json). The load_data() method reads this file and reconstructs all Participant and Event objects, restoring the system's complete state.

```
# JSON Data Structure stored in system_data.json
{
    'participants': {
        '3333': {
            'name': 'nischai', 'email': 'n@example.com',
            'mobile': '9999999999', 'p_id': 'N001',
            'evs': ['999020']
        }
    },
    'events': {
        '999020': {
            'name': 'jagmag', 'category': 'fest',
            'date': '2026-07-02', 'venue': 'niet old',
            'fee': 299.0
        }
    }
}
```

9.2 CSV-Based Report Export

The export_summary_csv() method uses Python's csv.writer to generate structured tabular reports. The CSV output contains one header row and one data row per event, formatted for direct use in spreadsheet applications like Microsoft Excel or Google Sheets.

```
# Sample CSV Output — event_summary_report.csv
Event ID,Event Name,Category,Scheduled Date,Registration Cost
bnmgvhdxf,fest,weddin,12,999.0
999020,jagmag,fest,2026-07-02,299.0
```

File	Format	Module Used	Data Stored	Purpose
data/system_data.json	JSON (nested dict)	json module	All participants and events	Cross-session persistence — save and restore system state
event_summary_report.csv	CSV (tabular rows)	csv module	Event ID, Name, Category, Date, Fee	Analytics export — downstream reporting and data analysis

CHAPTER 10: CONCLUSION & FUTURE SCOPE

10.1 Conclusion

The Event Management System (EMS) successfully demonstrates the comprehensive application of Python's Object-Oriented Programming paradigm in building a real-world, feature-rich application. The project effectively covers all major OOP principles — Abstraction through the Person ABC, Encapsulation via protected attributes, Inheritance in the Participant and Organizer class hierarchy, and Polymorphism through overridden display_details() methods.

Beyond basic OOP, the project showcases advanced Python features including decorators for automatic logging, generators for memory-efficient reporting, recursive functions for event searching, lambda expressions for sorting, list comprehensions for concise data filtering, and magic methods for operator overloading. The system's persistence layer — combining JSON for data storage and CSV for report exports — provides a complete, production-aware data management solution.

The Tkinter-based dark-themed GUI elevates the project beyond a basic console application, providing an intuitive, professional-grade interface with sidebar navigation, styled forms, and formatted output panels. All 15 test cases passed, confirming the reliability of the system's core functionality and exception handling mechanisms.

This project has provided the development team with valuable practical experience in software architecture design, Python programming best practices, GUI development, file I/O operations, and structured testing methodologies.

10.2 Future Scope

The Event Management System can be extended and enhanced in the following directions:

- Database Integration: Replace JSON file storage with a relational database (SQLite, PostgreSQL) or a NoSQL database (MongoDB) for scalability and concurrent multi-user access.
- Web Application: Convert the Tkinter desktop GUI into a full-stack web application using Flask or Django with a responsive HTML/CSS/JavaScript frontend.
- Email Notifications: Integrate Python's smtplib library to send automated confirmation emails and digital certificate PDFs to registered participants upon enrollment.
- QR Code Certificates: Generate QR code-embedded digital certificates using the qrcode library for tamper-proof certificate verification.
- Multi-user Authentication: Implement role-based access control (RBAC) with separate login accounts for Participants, Organizers, and Administrators.
- Payment Gateway Integration: Connect to real payment APIs (Razorpay, PayPal) for actual monetary transaction processing.
- Mobile Application: Develop a companion Android/iOS mobile app using Kivy or a React Native frontend connected to a Python REST API backend.
- Advanced Analytics: Add data visualization dashboards (matplotlib, seaborn, plotly) for graphical revenue analysis, attendance trends, and category distribution charts.

REFERENCES

1. van Rossum, G., & Drake, F. L. (2009). Python 3 Reference Manual. Scotts Valley, CA: CreateSpace.
2. Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media, Sebastopol, CA.
3. Ramalho, L. (2015). Fluent Python: Clear, Concise, and Effective Programming. O'Reilly Media.
4. Python Software Foundation. (2024). Python Documentation — Abstract Base Classes. https://docs.python.org/3/library/abc.html
5. Python Software Foundation. (2024). Python Documentation — json module. https://docs.python.org/3/library/json.html
6. Python Software Foundation. (2024). Python Documentation — csv module. https://docs.python.org/3/library/csv.html
7. Python Software Foundation. (2024). Python Documentation — tkinter module. https://docs.python.org/3/library/tkinter.html
8. Python Software Foundation. (2024). Python Documentation — datetime module. https://docs.python.org/3/library/datetime.html
9. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
10. Pilgrim, M. (2009). Dive Into Python 3. Apress.
11. Noida Institute of Engineering and Technology. (2025). Python Programming — Course Material, Department of CS & IT, NIET, Greater Noida.